Lab 4: Coalesced memory access and device (GPU) memory types

Phillip Pinter

March 22, 2024

1 Introduction

The goal of this lab session is split in two parts:

1. Evaluate the performance difference between coalesced and uncoalesced kernels. This is done by inverting the red color component in an image ($R = 255 - R$).
2. Evaluate the performance difference between global, shared and constant memory. This is done by multiplying two matrices.

These performance evaluations are done on different inputs (and therefore also grid sizes).

2 Results

2.1 Part1: Coalesced memory access

Coalesced memory access means that threads in a warp access the same memory group (adjacent addresses). When a thread accesses memory, it will also read data it doesn't necessarily needs since these accesses are done in blocks. Lets use fictional numbers to clarify this:
If a thread in a warp needs the data from address 0, addresses 0-3 will be read from memory since this is done in groups. If the next 3 threads in a warp happen to need addresses 1-3, there wouldn't be a need for additional memory accesses since that data is already read. This would be called coalesced memory access. If those next 3 threads didn't need the addresses 1-3, it would be clear that since those addresses are read but not used, the performance is impacted negatively.
To illustrate this, the red color component in an image is inverted in two different ways: firstly by using the original RGB format "RGBRGB...RGB" (uncoalesced) and secondly by reformatting the input to "RR...RGGG...GBB...B" (coalesced). This is done for two different images to illustrate different grid sizes (Lenna is a relativly small image of 512x512 pixels and VR is an image of 941x704). These tests were repeated multiple times to get an average time and this gave the following results:

```
Average time for Lenna image uncoalesced memory access: 0.0534787 ms
Average time for Lenna image coalesced   memory access: 0.0320861 ms
Average time for VR    image uncoalesced memory access: 0.0937918 ms
Average time for VR    image coalesced   memory access: 0.0383709 ms
```

These results show a significant improvement in performance by using coalesced memory access and this improvement is even bigger with the larger image. By using the "RR...RGGG...GBB...B" format, the threads in a warp all need to access the red components that are stored adjacently in memory which results in these performance improvements. Since the green and blue components aren't changed in this test, having them between the red components in the original format results in non-adjacent memory access for the threads in a warp which negatively impacts the performance.

2.2 Part2: Global, shared, and constant memory

To test the performance difference between global, shared and constant memory, multiplication of two matrices is implemented using those three memory types. Global storage is the easiest one to implement. Here you have memory that is globally accessible by all threads and this is the default memory type. Shared memory is memory that is shared among threads within a block. It's much faster than global memory because it's located on the chip itself, closer to the processor cores but the possible size of that memory can be quite limited, usually a few kilobytes per block. Constant memory is cashed on the GPU which makes accessing it very fast. The point of constant memory is to have fast read-only memory that threads need to use a lot. In the implementation on it, one of the two matrices is stored in constant memory since the input matrices don't need to change (so it can be read-only) and all the threads need data from it. Constant memory has a limited size as well so to make sure to have representative results, the matrices can't become to big so all three implementations can use the same input matrices. In this case they are limited to a size of 100x100 (which is used in these tests).

Since shared memory usually involves extra steps to implement, the time it takes to migrate memory from CPU to GPU global and form GPU global to GPU shared is timed. The time from CPU to GPU global is done by timing the time it takes to copy data from the host to the device. Assume the memory for the host is already allocated and initialized. So the timer starts when the device memory is made. After that the memory is allocated and the data from the host is loaded into the device. This is where the timer stops.
Timing from GPU global to GPU shared is trickier to do since this is done in the kernel itself. To do this, the kernel is timed like usual but the functionality of multiplying the two matrices is removed from the kernel so that essentially only the migration from global to shared memory remains. This gave the following results:

```
Memory migration from CPU to GPU global took: 0.501900000 miliseconds
Memory migration from GPU global to GPU shared took: 0.039935999 miliseconds
```

This shows that memory migration from CPU to GPU global is relatively slow compared to migration from GPU global to GPU shared. This is because in the second situation, the two locations are much closer to each other since they are on the same device. The first situation is slow because the data needs to go from the CPU all the way to the GPU which is a whole different device.

Performing the three different implementations gave the following results:

```
Average time for global memory: 0.078090951 miliseconds
Average time for shared memory: 0.114000022 miliseconds
Average time for constant memory: 0.059173245 miliseconds
```

This shows that constant memory is the fastest. Having fast read-only memory available for all the threads is beneficial for the performance since every thread needs data from this memory at every calculation so the faster the thread can read the data, the better. Since shared memory is closer to the threads and thus faster than global memory, the expected result would be that shared memory would be the fastest out of the two but the actual results show that shared memory is the lowest of all three. This could be due to the fact that shared memory requires additional steps and setup to handle the memory properly. In the results of the memory migration from GPU global to GPU shared it was visible that the migration alone took a substantial amount of time compared to the time it took to run the complete kernel with multiplication of the matrices. This shows that even tho shared memory itself is faster than global memory, the implementation isn't necessarily faster.

